

VTA Simulator: Comprehensive Architecture & Memory Integration Guide

1. Project Evolution: `main` vs. `development`

When this project started on the `main` branch, the `VTA-Simulator` was purely a **Timing Model**. The modules (`Load`, `Compute`, `Store`) contained empty `dependencies_received()` functions that simply called `finish.notify(latency())`. They simulated time passing, but no actual neural network math or data movement occurred. On the `development` branch, we upgraded the simulator into a **Functionally Accurate Hardware Model**. We accomplished two massive milestones:

- **Functional Math:** The `ComputeModule` now natively executes the TVM VTA GEMM (Matrix Multiplication) and ALU (Arithmetic Logic Unit) algorithms.
 - **AXI Hardware Integration:** The `LoadModule` and `StoreModule` now physically communicate with the 64KB Main Memory using strict, cycle-accurate AXI hardware handshakes.
-

2. Global Dataflow (End-to-End)

Here is the journey of a single neural network instruction from start to finish:

- **Parser:** The simulation boots, and `parser.cpp` reads the CSV file (e.g., `resnet_101.csv`), converting text into `Instruction` objects.
- **Fetcher:** Pushes the instructions into the three execution queues (`Load`, `Compute`, `Store`).
- **Dependency Queues (FIFOs):** If an instruction depends on a previous one (e.g., Compute needs Load to finish), it waits for a software token via the `l2c` (Load-to-Compute) FIFO queues.
- **Load Phase:** `LoadModule` triggers its AXI state machine. It negotiates the AXI bus and pulls 32-bit words from Main Memory into the 8-bit `inp_mem` and `wgt_mem` SRAMs.
- **Compute Phase:** `ComputeModule` triggers its math loops. It multiplies `inp_mem` with `wgt_mem`, accumulates the sum in `acc_mem`, and writes the 8-bit result to `out_mem`.

- **Store Phase:** `StoreModule` triggers its AXI state machine. It packs the 8-bit results from `out\_mem` into 32-bit words and negotiates the AXI bus to write them back to Main Memory.
-

3. The Hardware Bridge (Event-Driven to Clock-Driven)

The Problem: The VTA Simulator's base `Module` class is **event-driven**. Functions like `dependencies\_received()` trigger instantly and cannot use the `wait()` command. However, AXI memory relies on a **10ns clock** and requires `wait()` to synchronize signals. **The**

Solution: We created a "Bridge".

- We added `sc\_event start\_axi\_read` and `start\_axi\_write`.
 - We created independent `SC\_THREAD`s (`axi\_read\_thread` and `axi\_write\_thread`) that sleep in the background.
 - When `dependencies\_received()` triggers, it instantly fires the event, wakes up the AXI thread, and lets the thread take its time waiting on the clock cycles to do the heavy hardware lifting.
-

4. Module 1: `LoadModule` (The Data Fetcher)

Core Variables and Meaning:

- `sram\_base`: The starting index in the local SRAM array where data should be saved.
- `current\_address`: The physical address in DRAM we are reading from. Driven dynamically by `START\_ADDR`.
- `y\_size`, `x\_size`: The 2D dimensions of the data block being fetched.
- `stride`: How many bytes to jump in DRAM to get to the next row of the 2D block.
- `y0\_pad, y1\_pad, x0\_pad, x1\_pad`: Padding variables. Neural networks often require a border of zeros around an image/tensor before convolution.

Step-by-Step Logic (`axi\_read\_thread`):

- **Wait for Trigger:** Thread waits for `start\_axi\_read`.
- **Top Padding (`y0\_pad`):** Writes rows of pure `0`'s to the SRAM before fetching real data.
- **Row Loop (`y\_size`):**

- * **Left Padding (`x0\_pad`)**: Writes a few `0`s to start the row. * **AXI Address Phase**: * Sets `ARADDR = current\_address`. * Sets `ARLEN = 3` (Asking for 4 beats of data). * Sets `ARVALID = 1`. * *Handshake*: `do { wait(); } while (ARREADY == 0);` (Wait for memory to accept address). * **AXI Data Phase (The 4-Beat Chunking)**: * Because the memory is hardcoded for 4-beat bursts, we loop until `x\_size` is fulfilled, requesting 4 beats at a time. * *Delayed Handshake*: `RREADY` starts at `0`. We wait for the memory to assert `RVALID == 1`. *Only then* do we set `RREADY = 1` and `wait()` for one clock cycle. This prevents delta-cycle race conditions. * **Unpacking**: We read the 32-bit `RDATA`. We use bitshifts (`>> 0`, `>> 8`, `>> 16`, `>> 24`) and masks (`& 0xFF`) to split the 32-bit word into four 8-bit pieces. These are saved sequentially into `inp\_mem` or `wgt\_mem`. * **Right Padding (`x1\_pad`)**: Writes a few `0`s to end the row. * **Stride**: `current\_address += stride` moves the DRAM pointer to the next row.
 - **Bottom Padding (`y1\_pad`)**: Writes rows of pure `0`s at the end.
 - **Finish**: `finish.notify(latency())` tells the high-level simulator to pass the token to Compute.
-

5. Module 2: `ComputeModule` (The Math Engine)

Core Variables and Meaning:

- `inp\_mem`, `wgt\_mem`, `out\_mem`: The 8-bit globally shared SRAM arrays.
- `acc\_mem`: The 32-bit private accumulator SRAM.
- `uop\_mem`: The Micro-Op program SRAM. Holds instructions on *how* to stitch the tiles together.
- `DST\_MASK`, `SRC\_MASK`, `WGT\_MASK`: Bitmasks used to extract 11-bit and 10-bit indices from the 32-bit `uop` word.

Step-by-Step Logic (`dependencies\_received`):

- **Loop Setup**: Sets up the Outer Loop (`iter\_out`) and Inner Loop (`iter\_in`).
- **Micro-Op Loop**: Iterates through `uop\_mem` from `range\_0` to `range\_1`.
- **Decoding**: Grabs the 32-bit `uop` word.
 * `dst\_idx = (uop >> 0) & DST\_MASK` (Finds where to save the result). * `src\_idx` and `wgt\_idx` are extracted similarly.
- **GEMM Execution**:
 * Loops over `VTA\_BLOCK\_OUT` (16 output channels). * Reads the previous sum from `acc\_mem[dst\_idx]`. * Loops over `VTA\_BLOCK\_IN` (16 input channels). * **The Dot Product**:

Multiplies ``inp_mem * wgt_mem`` and adds it to ``tmp``. * Adds ``tmp`` to the accumulator. *
 Reset logic: If the instruction ``reset_out`` is true, the accumulator clears to 0 for the next run. *
 Quantization: Truncates the 32-bit sum down to an 8-bit integer (``& 0xFF``) and saves it to ``out_mem``.

- **ALU Execution:**

* If the instruction is ``ALU - max/min/add/shr``, it ignores weights. * It extracts ``src_0`` from the accumulator. * It extracts ``src_1`` (either from another accumulator tile, or a default immediate value ``imm``). * Performs element-wise math (e.g., ``result = src_0 + src_1``). * Truncates and saves to ``out_mem``.

6. Module 3: `StoreModule` (The Data Pusher)

Core Variables and Meaning:

- ``AWADDR``, ``AWVALID``, ``AWREADY``: AXI Address Write handshake pins.
- ``WDATA``, ``WVALID``, ``WLAST``, ``WREADY``: AXI Data Write handshake pins.
- ``BVALID``, ``BREADY``: AXI Write Response (confirmation) pins.

Step-by-Step Logic (`axi\_write\_thread`):

- **Wait for Trigger:** Waits for ``start_axi_write`` from the Compute module finishing.
 - **Row Loop (`y\_size`):**
 - * **AXI Address Phase:** Sets ``AWADDR`` and ``AWVALID = 1``. Waits for ``AWREADY == 1``. *
 - * **AXI Data Phase (The 4-Beat Chunking & Packing):** * Loops 4 times for the 4-beat burst.
 - * **Packing:** Reads four separate 8-bit integers from ``out_mem``. Shifts them left (``<< 0``, ``<< 8``, ``<< 16``, ``<< 24``) and uses binary OR (``|``) to pack them into a single 32-bit ``data_chunk``.
 - * Sets ``WDATA = data_chunk``. * Sets ``WLAST = 1`` on the 4th beat. * **Delayed Handshake:** * Waits for ``WREADY == 1`` *before* asserting ``WVALID = 1``. (Prevents the slave race-condition bug).
 - * **AXI Response Phase:** Sets ``BREADY = 1``. Waits for the memory to send ``BVALID == 1`` confirming the write succeeded to DRAM. * **Stride:** ``current_address += stride``.
 - **Finish:** ``finish.notify(latency())`` ends the instruction pipeline.
-

7. The Memory Integration Strategy (Why we bypassed the Arbiter)

During the Stage 2 AXI integration, we encountered a severe hardware deadlock with the teammate's ``axi_interconnect`` (Arbiter) module. **The Bug:** The teammate's Memory Slave state machine had a delta-cycle race condition. It asserted ``RLAST = 1`` (transfer finished), but instantly overwrote it to ``RLAST = 0`` in the exact same clock cycle. Because the Arbiter never saw ``RLAST``, it permanently locked the bus, assuming ``LoadModule`` was still using it, causing the entire simulator to freeze. **The Solution:** Our supervisor mandated that we must implement manual AXI interfaces and protocols, but stated that the teammate's code should not be altered. To satisfy both constraints, we wired the "Motherboard" (``vta.cpp``) intelligently:

- Instead of wiring Load and Store to the Arbiter, we wired them **directly to the Memory Slave (``axi_lite_slave``)**.
- Because ``LoadModule`` only reads (using the ``AR`` and ``R`` channels) and ``StoreModule`` only writes (using the ``AW``, ``W``, and ``B`` channels), they use entirely separate physical pins on the Memory chip. They cannot collide.
- This allowed us to maintain the strict, 100% accurate AXI ``VALID`/`READY`` handshakes inside our modulesâ€”fulfilling the educational requirements of the assignmentâ€”without being frozen by the bugged third-party Arbiter.